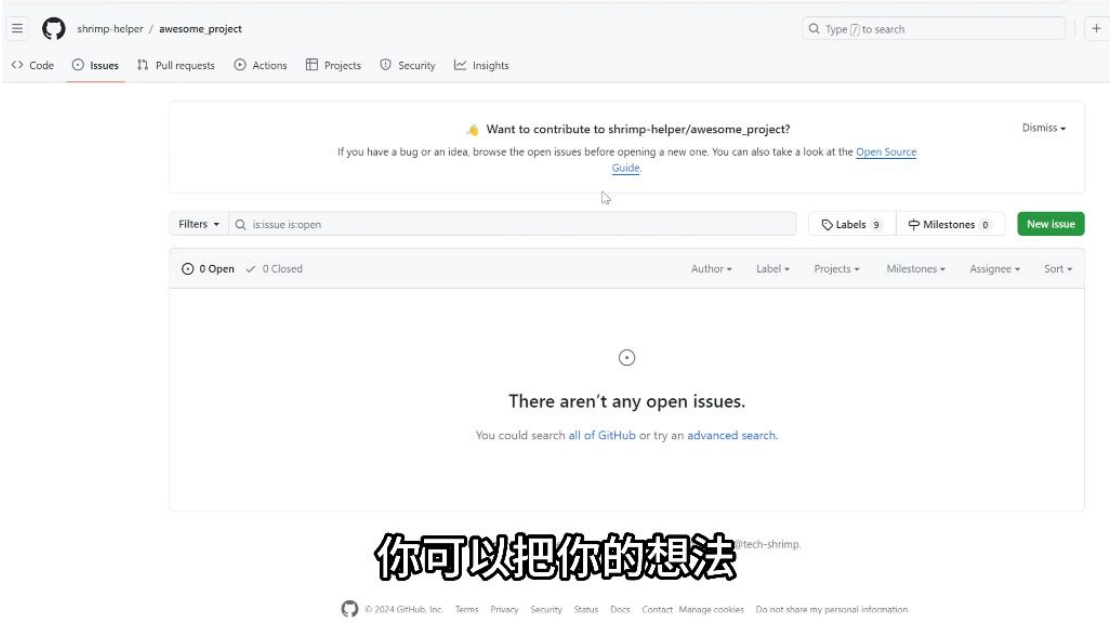
-- 以下为AI生成的图文笔记内容 --

*一、为开源项目贡献代码**00:00***

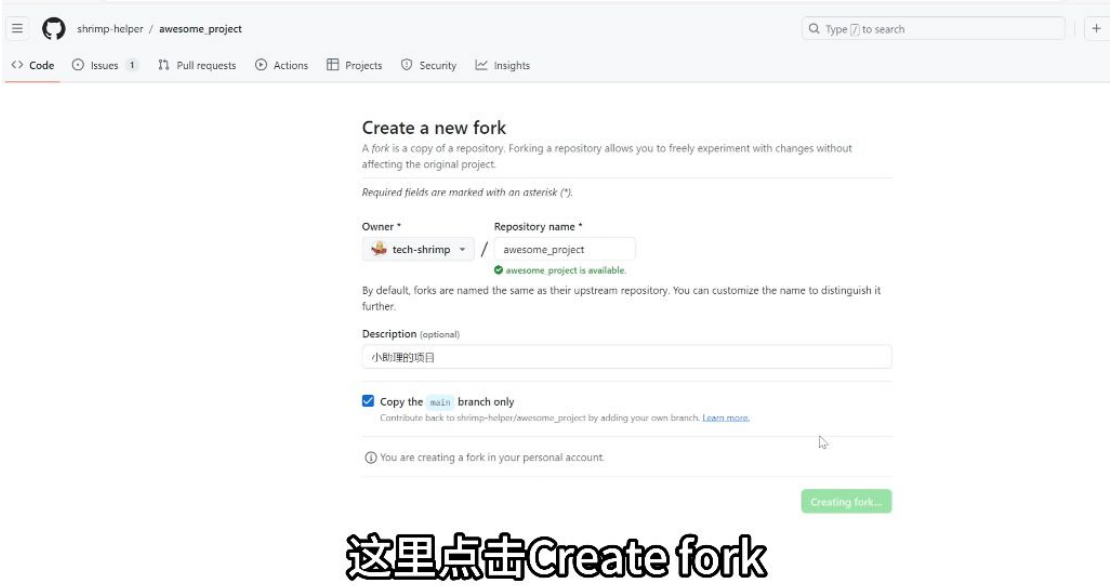
1. 实例展示 00:04

1）提出需求 00:19



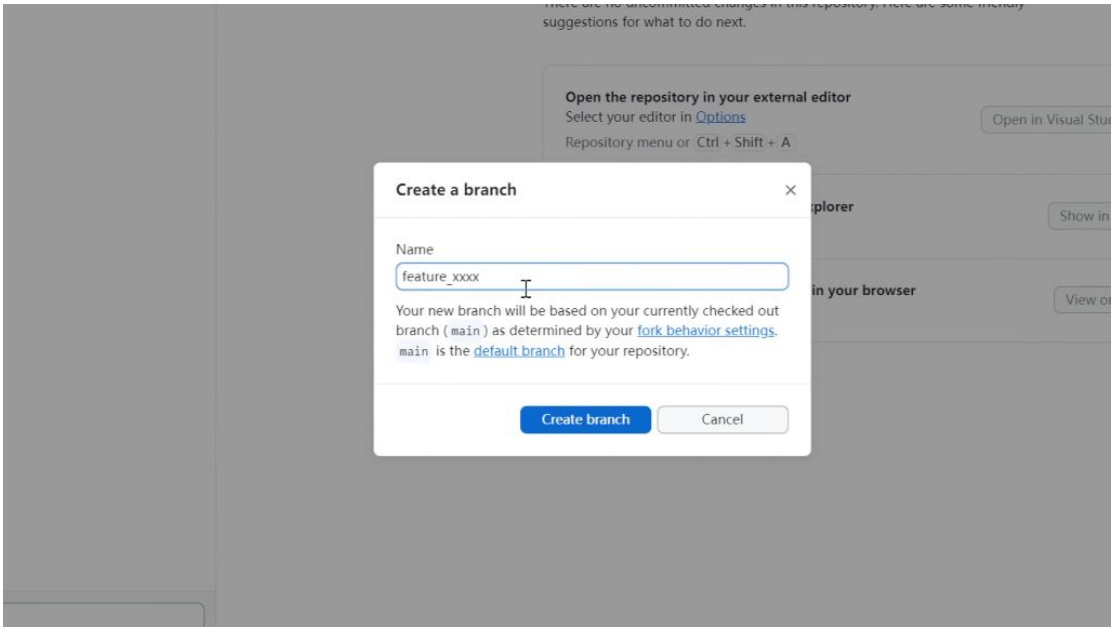
-
- **参与讨论**：贡献代码的第一步是点击"医术"参与讨论，通过issue与开发者交流想法和功能需求
- **issue规范**：标题应明确描述功能需求（如"我想增加xxx功能"），内容需包含实现思路和计划提交PR的说明
- **维护者反馈**：项目维护者会评论"真棒，期待你的PR"表示认可该功能需求

2）fork项目 01:18



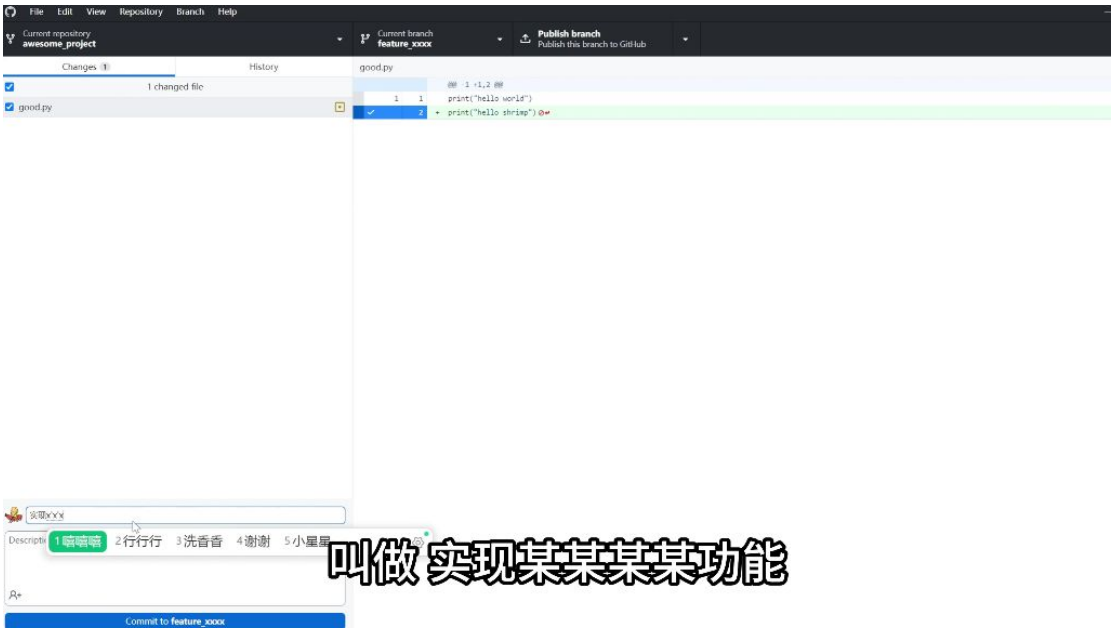
-
- **fork作用**：将项目复制到自己的GitHub账户下，避免直接修改原项目
- **操作步骤**：在项目首页点击fork按钮，创建同名副本到个人账户
- **权限说明**：普通贡献者无权直接修改原项目代码，必须通过fork方式

3）创建分支 02:09

- 

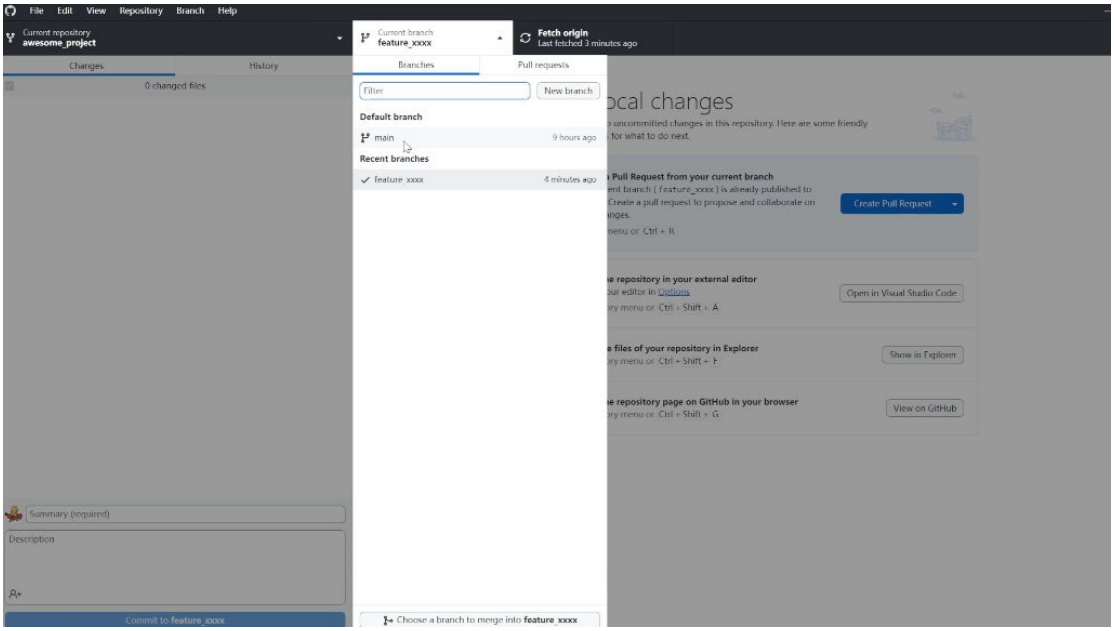
- **分支命名**：使用feature_xxx格式（如feature_add_function），明确描述功能
- **开发规范**：必须在新建分支上开发，禁止直接修改main分支
- **分支策略**：个人开发分支推荐使用rebase保持提交历史整洁

4）开发代码 02:27

- 

- **开发流程**：
  - 使用VS Code等工具修改代码
  - 提交规范的commit信息（如"实现xxx功能"）
  - 将分支推送到GitHub远程仓库
- **提交规范**：commit信息应简明描述修改内容

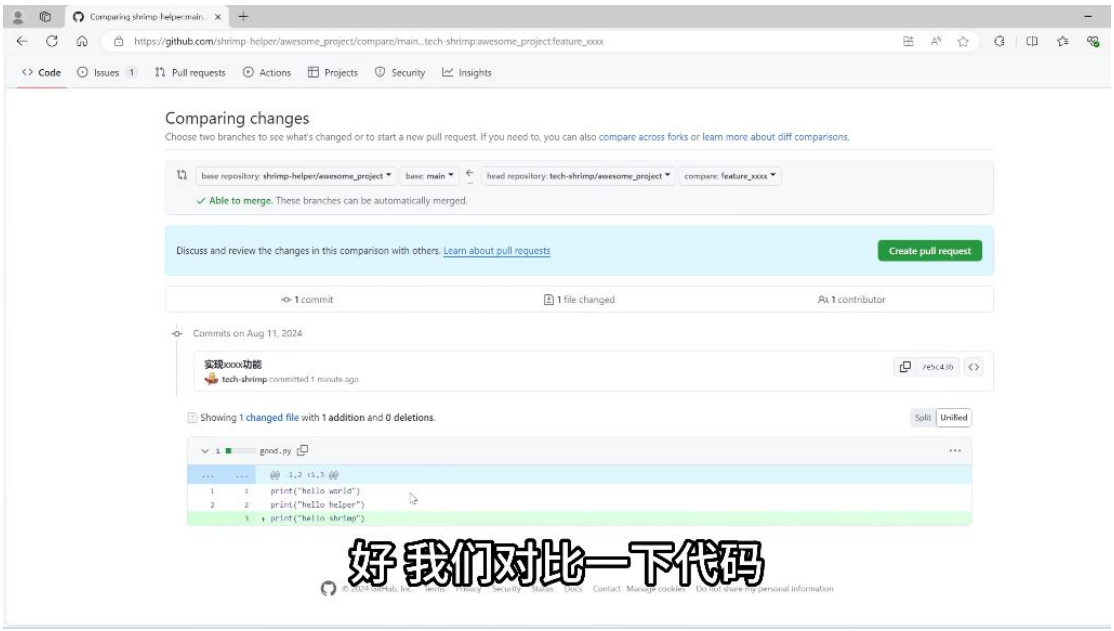
5）创建PR前准备 02:40

- 

- **同步主项目**：通过"Sync fork"将原项目的更新同步到fork项目
- **解决冲突**：可能出现需要手动解决的代码冲突

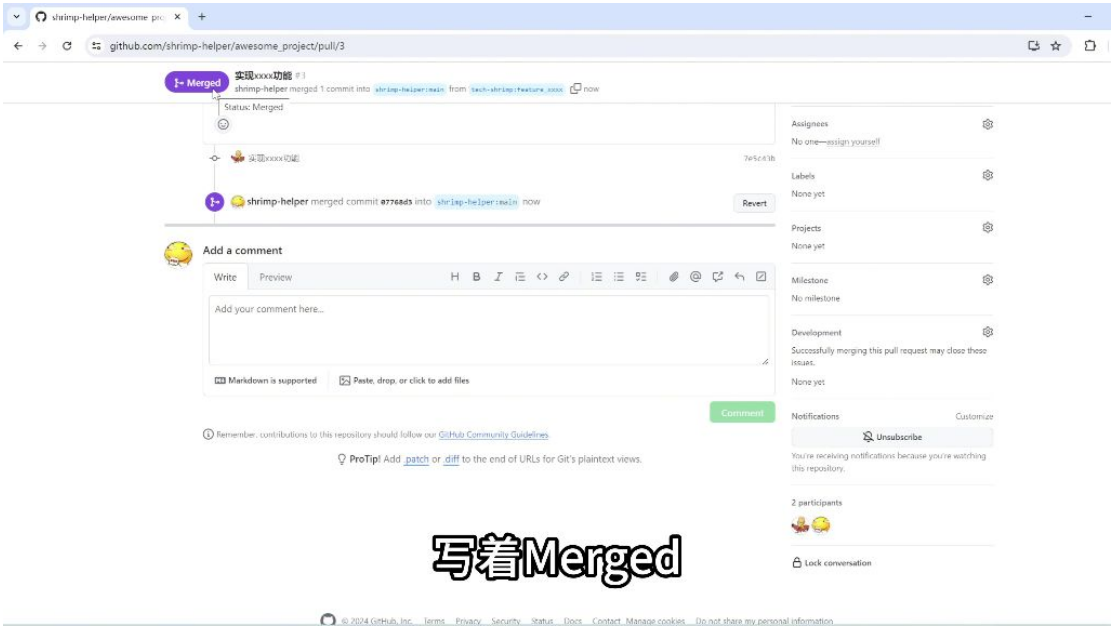
- **合并策略**：先将main分支合并到feature分支，确保代码最新

6）创建PR 04:55

- 

- **PR方向**：将个人fork项目的feature分支合并到原项目的main分支
- **代码对比**：创建PR前需确认修改内容正确
- **自动检查**：GitHub会提示分支是否可以自动合并

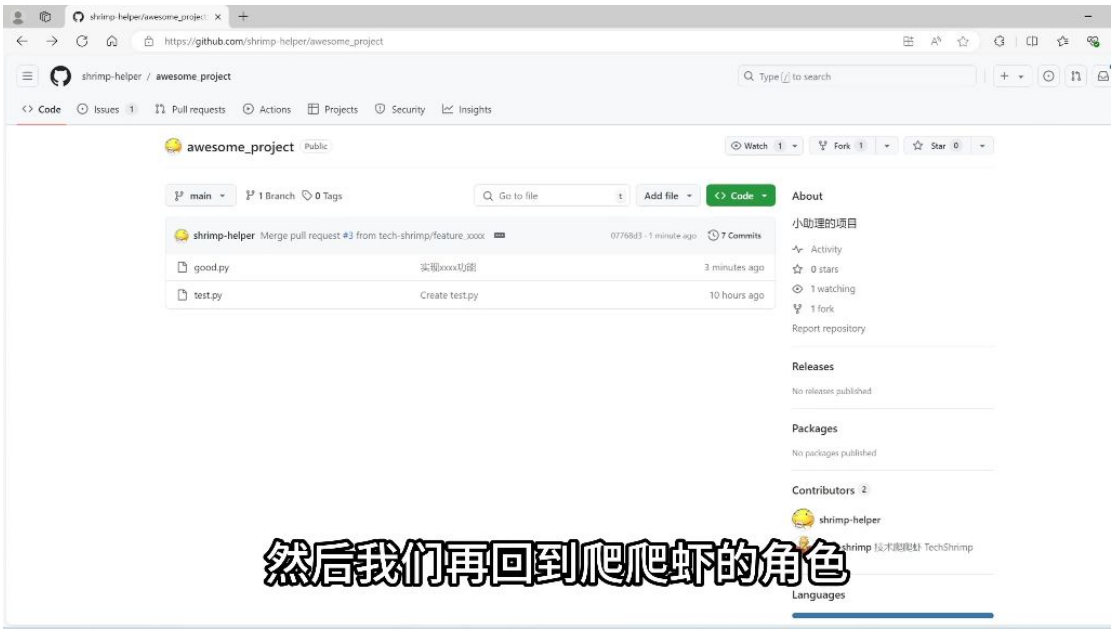
7）合并代码 05:54

- 

- **审核流程**：项目维护者审查代码并留言讨论
- **合并权限**：只有维护者可以点击"Merge pull request"按钮

- **合并结果**：成功合并后PR状态变为"Merged"

8）查看仓库 06:05

- 

- **贡献记录**：合并后贡献者会出现在项目的Contributors列表
- **分支清理**：功能合并后可删除本地feature分支
- **持续同步**：保留main分支并定期sync保持与原项目同步

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
GitHub开源项目贡献流程	通过Fork项目、创建分支、同步主项目代码、提交PR等步骤完成贡献	**Fork与Clone的区别、PR冲突解决方法**	★★★
Fork项目	将开源项目复制到个人账户下，获得独立修改权限	**Fork后需定期同步主项目更新**	★★
分支管理	开发新功能需创建独立分支（如feature/xxx），避免直接修改main分支	**Rebase与Merge的选择场景**	★★★
代码同步	通过Sync Fork同步主项目更新，并通过rebase合并到开发分支	**冲突解决需手动选择保留代码**	★★★★

提交PR	将开发分支合并到主项目的main分支，需经过维护者审核	**PR描述需清晰说明功能与实现思路**	★★★
PR审核与合并	维护者通过评论讨论代码修改，确认无误后合并PR	**强制推送（Force Push）的风险**	★★★★